

Obiecte si clase

Definirea propriilor clase

- Forma standard a definiției clasei în Java arată astfel:

```
public class name extends superclass {  
    class body  
}
```

Fiecare clasă publică trebuie stocată într-un fișier separat al cărui nume este numele clasei urmat de **.java**.

- Clauza **extends** de pe linia de antet specifică numele superclasei. Dacă clauza **extends** lipsește, noua clasă devine o subclasă directă a obiectului, care este rădăcina ierarhiei de clase Java
- Corpul unei clase este format dintr-un set de definiții Java care sunt denumite generic *intrări*. Cele mai frecvente intrări sunt constructori, metode, variabile de instanță și constante

Reprezentarea informațiilor despre Studenti

- Înțelegerea structurii unei clase este mai ușoară în contextul unui exemplu specific. În continuare se arată definiția unei clase numită **Student**, care este utilizată pentru a urmări următoarele informații despre un student :
 - Numele studentului
 - Numărul de identificare din șase cifre al elevului
 - Numărul de credite pe care le-a obținut studentul
 - Un indicator care indică dacă studentul a plătit toate taxele universitare
- Fiecare dintre aceste valori este stocată într-o variabilă de instanță de tipul corespunzător.
- În conformitate cu convenția modernă orientată obiect, aceste variabile de instanță sunt declarate **private**. Prin urmare, accesul la aceste valori este intermediat de metode din clasa Student.

Clasa Student

Acest comentariu descrie clasa în ansamblu.

```
/**
 * Clasa Student ține evidența următoarelor date despre un student
 * numele studentului, numărul de identificare,
 * numărul de credite obținute de student
 * și, dacă studentul are platite toate taxele.
 * Toate aceste informații sunt complet private pentru clasă.
 * alte clase pot obține aceste informații numai utilizând diverse
 * metode definite de clasă.
 */
```

```
public class Student {
```

*Antetul clasei definește **Student** ca o subclasă directă a clasei **Object***

```
/**
 * Creează un nou obiect Student cu numele și ID-ul specificat.
 * @param name este numele studentului ca șir
 * @param ID este numărul de identificare al elevului ca int
 */
```

```
    public Student(String name, int id) {
        numeStudent = name;
        studentID = id;
    }
```

Acest comentariu descrie constructorul.

Constructorul actualizează variabilele de instanță.

Clasa Student

```
/**
 * Furnizeaza numele acestui student.
 * @return Numele acestui student
 */
public String getName() {
    return numeStudent;
}
```

Aceste metode furnizeaza valoarea unei variabile de instanță și se numesc **getters**. Deoarece numele studentului și numărul de identificare sunt fixate, nu există **setters** corespunzători.

```
/**
 * Furnizeaza ID ul acestui student.
 * @return ID ul acestui student
 */
public int getID() {
    return studentID;
}
```

Această metodă modifică valoarea unei variabile de instanță și se numește **setter**.

```
/**
 * Actualizeaza (seteaza) numarul de credite.
 * @param credite Noul număr de unități câștigate
 */
public void setCredite(int credite) {
    crediteObtinue = credite;
}
```

Clasa Student

```
/**
 * Furnizeaza creditele obtinute de acest student.
 * @return Numărul de credite pe care le-a obtinut acest student
 */
public int getCredite() {
    return crediteObtinue;
}

/**
 * actualizeaza informatia referitoare la plata taxelor.
 * @param flag poate fi true sau false care indică starea achitată
 */
public void setPlatit(boolean flag) {
    platit = flag;
}

/**
 * Furnizeaza informatia despre plata taxelor
 * @return daca studentul a platit
 */
public boolean estePlatit() {
    return platit;
}
```

Numele pentru metodele getter încep de obicei cu prefixul **get**. Singura excepție este pentru metodele getter care returnează un **boolean**, caz în care numele începe de obicei cu **is**.

Clasa Student

```
/**
 * Creează un șir care identifică acest student.
 * @return Șirul folosit pentru afișarea acestui student
 */
public String toString() {
    return numeStudent + " (" + studentID + ")";
}
```

Metoda **toString** spune mediului Java cum să afișeze o valoare a acestei clase. Toate clasele ar trebui să suprascrie (override) **toString**

```
/* Constante Public */
```

Clasele exportă adesea constante

```
/** Numărul de credite necesare pentru absolvire */
public static final int CREDITE_NECESARE_ABSOLVIRE = 180;
```

```
/* Variabile de instanța private */
```

```
private String numeStudent; /* Numele studentului */
private int studentID; /* ID student */
private int crediteObtinite; /* Numar de credite obtinute */
private boolean platit; /* Daca studentul a platit taxe */
```

```
}
```

Aceste declarații definesc variabilele de instanță care mențin starea internă a clasei. Toate variabilele de instanță utilizate sunt private.

Utilizarea clasei Student

- După ce ați definit clasa Student, puteți utiliza apoi constructorul acesteia pentru a crea instanțe ale acelei clase. De exemplu, puteți utiliza următorul cod pentru a crea două obiecte Student:

```
Student sefLiga = new Student("Alex Popa", 123456);  
Student sefPromotie = new Student("Alina Giurgiu", 314159);
```

- Se pot utiliza oricare dintre metodele prezentate anterior cu aceste doua instanțe ale clasei Student. De ex pentru a actualiza numărul de credite , respectiv pentru a afla numele

```
sefPromotie.setCredite(263);
```

```
sefLiga.getName();
```


Numere raționale

- Ca un exemplu mai elaborat de definiție a unei clase, o clasă numită **Rational** care reprezintă *numere raționale*, care sunt pur și simplu două numere întregi.
- Numerele raționale pot fi utile în cazurile în care aveți nevoie de calcul exact cu fracții. Chiar dacă utilizați un dublu, numărul în virgulă mobilă 0,1 este reprezentat intern ca o aproximare. Numărul rațional 1/10 este exact.
- Numerele raționale acceptă operațiile aritmetice standard :

Adunare:

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}$$

Scadere:

$$\frac{a}{b} - \frac{c}{d} = \frac{ad - bc}{bd}$$

Înmulțire:

$$\frac{a}{b} \times \frac{c}{d} = \frac{ac}{bd}$$

Împartire:

$$\frac{a}{b} \div \frac{c}{d} = \frac{ad}{bc}$$

Implementarea clasei **Rational**

- Pe măsură ce citiți codul, următoarele caracteristici merită o atenție specială :
 - *Constructorii clasei sunt supraîncărcați.* Apelarea constructorului fără argument creează un număr rațional inițializat la 0, apelarea acestuia cu un argument creează un rațional egal cu acel număr întreg și apelarea acestuia cu două argumente creează o fracție
 - *Constructorul se asigură că numărul este redus la termenii cei mai mici.* Mai mult, deoarece aceste valori nu se schimbă niciodată odată cu crearea unui nou rațional, această proprietate va rămâne în vigoare.
 - *Operations are specified using the receiver syntax.* When you apply an operator to two **Rational** values, one of the operands is the receiver and the other is passed as an argument, as in

r1.add(r2)

Cuvântul cheie **this**

- Implementarea clasei **Rational** folosește cuvântul cheie Java **this**, care se referă întotdeauna la obiectul curent. Cu toate acestea, codul îl folosește în două moduri distincte :
 1. În primele două versiuni ale constructorului, cuvântul cheie acesta este folosit pentru a invoca unul dintre ceilalți constructori pentru această clasă.
 2. În metodele care implementează operatorii aritmetici, aceasta este utilizată pentru a sublinia faptul că un câmp numit provine din acest obiect mai degrabă decât din una din celelalte valori ale acelei clase.
- Cuvântul cheie **this** este de asemenea folosit adesea pentru a distinge între o variabilă de instanță și o variabilă locală cu același nume. Dacă includeți **this** înainte de numele variabilei, indicați compilatorului că vrei să spui variabila de instanță.

The Rational Class

```
/**
 * Clasa Rational este utilizată pentru numere raționale,
 * definite ca fiind raportul a două numere întregi.
 */
public class Rational {

    /** Creeaza un nou Rational initializat cu zero. */
    public Rational() {
        this(0);
    }

    /**
     * Creaza un rational cu numitor 1.
     * @param n valoare numarator
     */
    public Rational(int n) {
        this(n, 1);
    }
}
```

*These constructors are overloaded so that there is more than one way to create a **Rational** value. These two versions invoke the primary constructor by using the keyword **this**.*

The Rational Class

```
/**
 * Creeaza un Rational de valoare x / y.
 * @param x numarator
 * @param y numitor
 */
public Rational(int x, int y) {
    int g = cmmdc(Math.abs(x), Math.abs(y));
    num = x / g;
    den = Math.abs(y) / g;
    if (y < 0) num = -num;
}
```

Constructorul primar creează un nou **Rational** pornind de la numărător și numitor. Apelul la `cmmdc` asigură că fracția este redusă la termenii cei mai mici.

Metoda **add** creează un nou obiect **Rational** folosind regula de adunare. Cele două valori raționale apar în **this** și **r**.

```
/**
 * Aduna numarul rational r la acest numar.
 * @param r numarul care se aduna
 * @return suma dintre numarul curent si r
 */
public Rational add(Rational r) {
    return new Rational(this.num * r.den + r.num * this.den,
                        this.den * r.den);
}
```

The Rational Class

```
/**
 * Scade numarul rational r din acest numar.
 * @param r este numarul care se va scadea
 * @return rezultatul scaderii
 */
public Rational subtract(Rational r) {
    return new Rational(this.num * r.den - r.num * this.den,
                        this.den * r.den);
}

/**
 * Multiplica cu r numarul rational
 * @param r Numarul pentru inmultire
 * @return Rezultatul inmultirii
 */
public Rational multiply(Rational r) {
    return new Rational(this.num * r.num, this.den * r.den);
}
```

Aceste metode (împreună cu **divide** din pagina următoare) funcționează la fel ca metoda **add**, dar utilizează o formulă diferită. Rețineți că aceste metode au acces la componentele *r*.

The Rational Class

```
/**
 * imparte numarul rational curent cu r.
 * @param r numarul utilizat ca divizor
 * @return rezultatul impartirii cu r
 */
public Rational divide(Rational r) {
    return new Rational(this.num * r.den, this.den * r.num);
}

/**
 * Creeaza o reprezentare ca String.
 * @return reprezentarea ca String
 */
public String toString() {
    if (den == 1) {
        return "" + num;
    } else {
        return num + "/" + den;
    }
}
```

Această metodă convertește numărul rațional în forma de șir. Dacă numitorul este 1, numărul este afișat ca număr întreg.

The Rational Class

```
/**  
 * Calculeaza cmmdc pe baza algoritmului lui euclid.  
 * @param x primul intreg  
 * @param y al doilea intreg  
 * @returneaza cmmdc al lui x si y  
 */
```

```
private int cmmdc(int x, int y) {  
    int r = x % y;  
    while (r != 0) {  
        x = y;  
        y = r;  
        r = x % y;  
    }  
    return y;  
}
```

Metoda **cmmdc** este declarată privată deoarece face parte din implementarea acestei clase și nu este folosită niciodată în afara acesteia.

```
/* Variabile private de instanta */  
private int num;    /* numarator Rational */  
private int den;    /* numitor Rational */
```

```
}
```

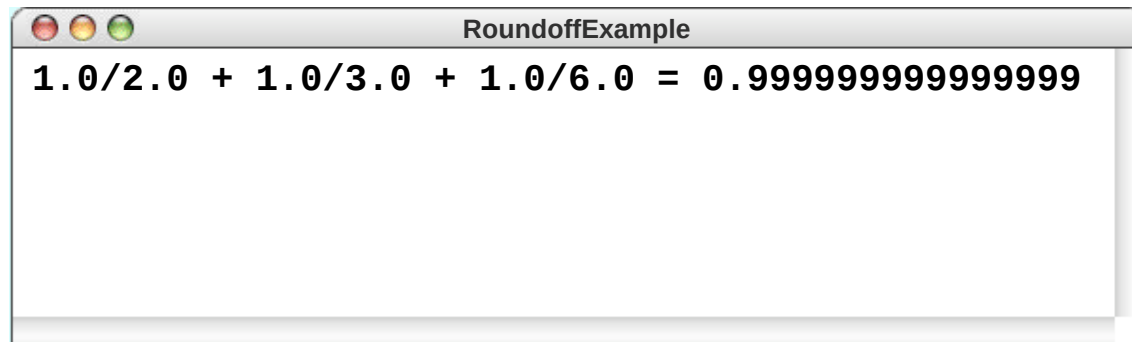
Ca întotdeauna, variabilele de instanță sunt private

Simularea calculului rațional

- In continuare se arata modul de calcul al sumei.
-

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{6}$$

- Cu aritmetica rațională, calculul este exact. Dacă scrieți același program folosind variabile de tip **double**, rezultatul arată astfel:



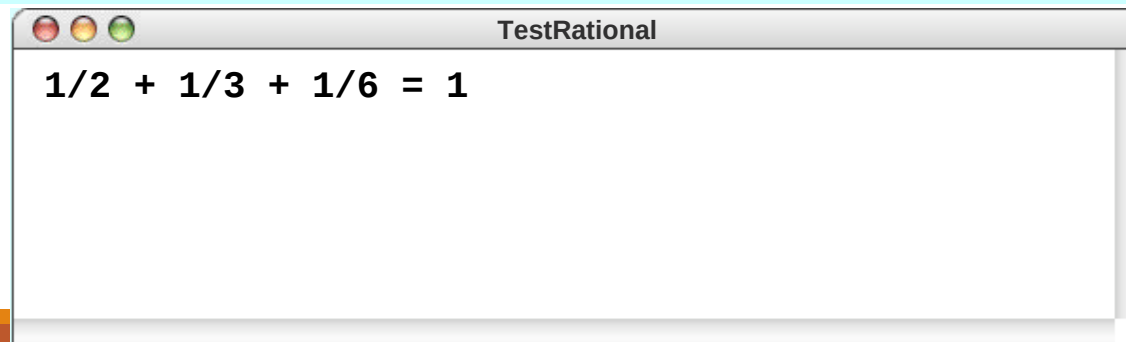
```
RoundoffExample
1.0/2.0 + 1.0/3.0 + 1.0/6.0 = 0.9999999999999999
```

The image shows a screenshot of a Java Swing window titled "RoundoffExample". The window has a standard macOS-style title bar with red, yellow, and green buttons. The content of the window is a single line of text: "1.0/2.0 + 1.0/3.0 + 1.0/6.0 = 0.9999999999999999". This demonstrates the result of a floating-point calculation using the `double` type, which is not exact due to rounding.

Adunarea a trei valori raționale

```
public class TestRational {  
    public static void main (String[] args) {  
        Rational a = new Rational(1, 2);  
        Rational b = new Rational(1, 3);  
        Rational c = new Rational(1, 6);  
        Rational sum = a.add(b).add(c);  
        System.out.println(a + " + " + b + " + " + c + " = " + sum);  
    }  
}
```

a	b	c	sum
1 2	1 3	1 6	1 1



Utilizarea claselor din pachetul java.lang

Clasa Object

Clasa Object este rădăcina ierarhiei de clase a platformei Java

Orice altă clasă este derivată direct sau indirect din clasa Object.

Metodele clasei Object

- Clasa are un singur constructor, `Object()`,
- `public boolean equals(Object obj)`
- **`public String toString()`**
- **`protected Object clone()`**
- `b=a.clone();`

Clasele String și StringBuffer

Toate șirurile în limbajul Java, inclusiv literalii-șir, de exemplu "ABCdef123" sunt obiecte din String

- constructorul String(String s).
- int compareTo(String anotherString)
- int compareToIgnoreCase(String str)
- int length()
- String trim()
- int indexOf(int ch)
- int indexOf(int ch, int fromIndex)
- int indexOf(String str)
- int indexOf(String str, int fromIndex)
- int lastIndexOf(int ch)
- int lastIndexOf(int ch, int fromIndex)
- String substring(int beginIndex)
- String substring(int beginIndex, int endIndex)

Exemplu de program:

```
class TestStr {  
  
    public static void main(String args[]) {  
  
        String s1="ABCdef##&&", s2, s3, s4;  
  
        char ch1[]; byte tb[];  
  
        s2=new String("Un sir nou");  
  
        System.out.println("s1="+s1+" s2="+s2);  
  
        System.out.println("Lungimile sirurilor: s1="+s1.length()+ " s2="+s2.length());  
  
        System.out.println("Conversie in litere mari: s1="+s1.toUpperCase()+ " s2="+s2.toUpperCase());  
  
        System.out.println("Conversie in litere mici: s1="+s1.toLowerCase()+ " s2="+s2.toLowerCase());  
  
        ch1=s1.toCharArray();  
  
        System.out.print("s1 convertit in tablou de caractere: ");  
  
        for(int i=0; i <ch1.length;i++)System.out.println(ch1[i]);  
  
        System.out.println;  
  
        tb=s1.getBytes();  
  
        System.out.print("s1 convertit in tablou de octeti: ");
```

```
for(int i=0; i <tb.length;i++) System.out.print(tb[i]+" ");

System.out.print("\nAcelasi tablou convertit in caractere: ");

for(int i=0; i <tb.length; i++) System.out.print((char)tb[i]);

System.out.println;

s3=String.valueOf(ch1);

System.out.println("Sirul obtinut din tabloul de caractere: "+s3);

System.out.println("Comparatii de siruri: "+s1.compareTo(s3)+" "+

s1.compareTo(s2)+" "+s2.compareTo(s1));

System.out.println("s1==s3: "+(s1==s3)+" s1.equals(s3): "+ s1.equals(s3));

s4=s2.toUpperCase();

System.out.println("s2="+s2+" s4="+s4);

System.out.println("s2.equals(s4): "+s2.equals(s4)+" s2.equalsIgnoreCase(s4): "+s2.equalsIgnoreCase(s4));

System.out.println("Subsirul lui s2 intre pozitiile 3 si 8: "+ s2.substring(3,8));

System.out.println("Subsirul lui s2 de la pozitia 3 la sfarsit: "+ s2.substring(3)); }

}
```

Clasa Class

Câteva dintre metodele clasei Class:

`public String getName()` - întoarce numele calificat al unei entități (clase, interfețe, tip primitiv) reprezentată de un obiect din clasa Class;

`public boolean isAssignableFrom(Class cls)` - întoarce true dacă clasa căreia i se aplică metoda este o superclasă a clasei cls, primită ca argument;

`public boolean isInterface()` - întoarce true dacă metoda este aplicată unei instanțe a clasei Class care reprezintă o interfață; `public boolean isPrimitive()` - întoarce true dacă metoda este aplicată unui obiect din clasa Class care reprezintă un tip de date primitiv;

`public Class getSuperclass()` - întoarce o instanță a clasei Class care reprezintă superclasa obiectului căruia i se aplică această metodă.

În fișierul `TestClass.java` este dat un program de testare a obținerii obiectelor Class și a aplicării unora dintre metodele acestora. Descrierea completă a clasei Class este dată în documentația Java API.

```
/* Testarea clasei Class */
class TestClass {
public static void main(String args[]) {
String s1="Primul sir", s2="al doilea sir";
Object obj=new Object();
Class cls1, cls2, cls3;
cls1=s1.getClass();
System.out.println(cls1.getName());
System.out.println(cls1.toString());
cls2=s2.getClass();
System.out.println(cls1.isAssignableFrom(cls2));
cls3=obj.getClass();
System.out.println(cls3); System.out.println(cls1.isAssignableFrom(cls3));
System.out.println(cls3.isAssignableFrom(cls1)); System.out.println(cls1.getSuperclass());
System.out.println(cls3.getSuperclass()); System.out.println(cls1.isPrimitive());
}
}
```

Clasele acoperitoare de tip

În pachetul `java.lang`, pentru fiecare din tipurile de date primitive există o clasă acoperitoare de tip (engleză: Type Wrapper). Aceste clase pun la dispoziția programatorului atât variabile finale (constante), cât și metode necesare pentru a lucra cu tipul de date respectiv. Clasele acoperitoare

Figura 1.

